
UNIT 8 THREAD HANDLING IN PYTHON

Structure

8.0 Introduction

8.1 Objectives

8.2 Understanding Threads

8.2.1 Thread vs. Process

8.2.2 Benefits of Using Threads

8.2.3 Thread Lifecycle

8.2.4 Limitations in Python: The GIL

8.2.5 Thread Use Cases

8.3 Python Threading Module

8.3.1 Creating and Starting a Thread

8.3.2 Thread Naming

8.3.3 Thread Identifiers

8.3.4 Useful Thread Functions

8.3.5 The join() Method

8.3.6 Daemon Threads

8.4 Thread Synchronization

8.4.1 Race Condition

8.4.2 Locks (Mutexes)

8.4.3 Reentrant Locks (RLock)

8.4.4 Condition Variables

8.4.5 Semaphores

8.5 Advanced Threading Techniques

8.5.1 Thread Pools

8.5.2 Daemon vs. Non-Daemon Threads

8.5.3 Thread Communication with Queue

8.5.4 Timer Threads

8.6 Multithreading with Real-world Examples

8.6.1 Example 1 – Web Scraping (Concurrent Requests)

8.6.2 Example 2 – Producer–Consumer Problem

8.6.3 Example 3 – Logging in Background

8.6.4 Example 4 – Simulation (Bank Transactions with Lock)

8.7 Summary

8.8 Solutions/Answers to Check Your Progress

8.9 References/Further Readings

8.0 INTRODUCTION

In many real-world applications, simultaneously handling multiple tasks is critical for both performance and resource utilization. For example, a browser may download a file while allowing the user to browse other pages or play media. Threading makes this possible: each thread represents an independent sequence of execution within a single process.

Threads allow multiple tasks to run in parallel on one or more CPU cores by representing independent sequences of execution within a process. This lets

programs run parallel tasks efficiently, since all threads within a process share the same memory. This improves system resource utilization and is especially beneficial in developing network-based, gaming, and web-based applications.

However, this also introduces challenges such as race conditions, when multiple threads attempt to modify shared data simultaneously. In this unit, you will learn how to implement multithreading in Python.

8.1 OBJECTIVES

By the end of this unit, you will be able to:

- Explain what a thread is and how it differs from a process,
- Create, start, and manage threads using Python's threading module,
- Synchronize access to shared resources with Locks, Semaphores, and Conditions,
- Use higher-level patterns like thread pools and daemon threads for cleaner design, and
- Apply multithreading to practical problems (e.g., web scraping, producer-consumer workflows).

8.2 UNDERSTANDING THREADS

A process is an instance of a running program, while a thread is the smallest unit of execution within a process. In Python, the operating system manages the underlying process through its interpreter, which executes Python bytecode. Python supports multithreading, which allows multiple threads to run concurrently within a single process. The main interpreter process always starts with a main thread responsible for running the program. Additional threads can be created to execute code concurrently with the main thread.

Each thread has:

- Its own stack (execution context).
- Access to CPU registers.
- Shared memory space with other threads in the process.

While shared memory allows fast communication between threads, it also raises the risk of data conflicts.

Imagine multiple users collaboratively editing a shared spreadsheet. If they edit different cells, everything works fine. But if two users edit the same cell simultaneously, errors may occur unless proper coordination is used.

8.2.1 Thread vs. Process

A process has its own memory space and system resources, and a thread runs inside a process and shares its memory.

Feature	Process	Thread
Memory	Independent address space	Shared within the process
Communication	Inter-process communication (sockets, pipes)	Direct access to shared data
Overhead	Higher (expensive context switching)	Lower (lightweight switching)
Isolation	Strong (independent, safer)	Weak (requires synchronization)

In Python, threads run inside a process and share memory, making them ideal for **I/O-bound tasks** like network requests, file operations, and database queries that benefit from concurrency.

8.2.2 Benefits of Using Threads

The use of threads in application development contributes significantly to improving responsiveness, operational efficiency, and design scalability. Key benefits include:

- **Responsiveness:** Threads offer a smooth user experience where background tasks can be executed without interrupting the user interface.
- **Parallelism:** Threads are most useful in I/O-bound operations where multiple tasks can be executed concurrently, such as file access or network communication.
- **Optimized Resource Utilization:** Threads offer better CPU utilization by minimizing idle time for the tasks waiting for external resources.

While threading brings notable advantages, it also introduces challenges:

- **Race Conditions:** When two or more threads access shared resources simultaneously without proper synchronization, it may lead to incorrect results.
- **Deadlocks:** When two or more threads wait indefinitely for resources held by each other, potentially halting execution and none of them can proceed.
- **Starvation:** When a thread restarts many times and never gets CPU access due to scheduling or locks.
- **Global Interpreter Lock (GIL):** The GIL restricts executing one thread at a time, which implies no true parallelism for CPU-bound operations.

8.2.3 Thread Lifecycle

The lifecycle of a thread refers to the various states from creation to termination:

1. **New** – Created but not yet started.

2. **Runnable** – Ready to run, waiting for CPU time.
3. **Running** – Actively executing.
4. **Blocked/Waiting** – Suspended, waiting for I/O or lock release.
5. **Terminated** – Completed or exited.

Lifecycle Diagram:

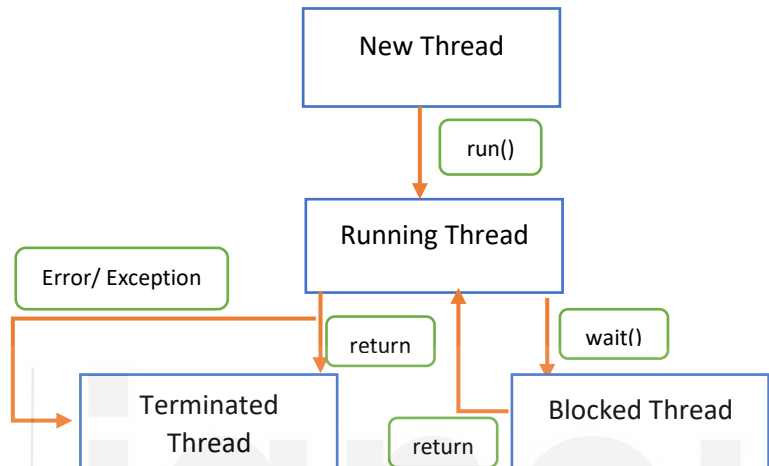


Figure 1: Lifecycle of a Python thread

8.2.4 Limitations in Python: The Global Interpreter Lock (GIL)

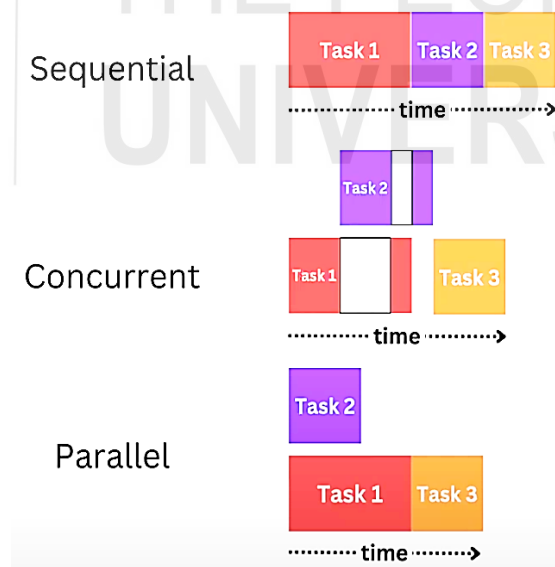


Figure 2: Task Execution

Consider the sequential execution for three tasks, where all tasks are executing in order. In concurrency, the processor starts on task one, and then at some point in time it switches to execute task two. Switching back and forth and interleaving across tasks means using the downtime of one task to do something else in the

meantime. In parallelism, it is much simpler: the two tasks execute at the same time. The sequential execution is slowest, and concurrency is slightly faster in this example, and parallelism is the fastest.

The GIL ensures only one thread executes Python bytecode at a time, even on multicore processors. This restriction means Python threads do not run in true parallel for CPU-bound tasks in multi-threaded Python programs. For I/O-bound tasks (e.g. file I/O, network calls), threads still offer significant performance gains. To fully leverage multiple cores for CPU-heavy tasks, developers often turn to multiprocessing.

Python uses the Global Interpreter Lock (GIL) to ensure only one thread executes Python bytecode at a time.

- For **I/O-bound operations**, threading is still effective since threads spend time waiting for external resources (network, disk).
- For **CPU-bound operations**, threading does **not** improve performance due to the GIL. Instead, **multiprocessing** is recommended.

8.2.5 Thread Use Cases

Multithreading is particularly effective in these areas:

- **GUI applications:** Keep UI responsive while long-running tasks occur in the background.
- **Network servers:** Handle multiple clients concurrently.
- **Web scraping:** Fetch multiple pages at once.
- **Logging systems:** Record events while processing data.
- **Simulations/games:** Manage multiple agents in parallel.

Many real-life applications—from gaming systems to online banking platforms—rely on multithreading to deliver seamless performance.

8.3 PYTHON THREADING MODULE

Python provides the threading module for thread management. By default, when a Python script runs, the main thread executes. Additional worker threads can be created using the Thread class.

8.3.1 Creating and Starting a Thread

The Python standard library provides the threading module. Whenever you run any Python script, the Python interpreter requests the operating system to create one thread (the default main thread). The threads are Python objects of threading, and threading is a module with one Thread class. To start a separate thread, you create a Thread instance and then tell it to start ():

```
from threading import Thread, current_thread
#Create a function containing code to be executed parallelly
def display (n, msg):
```

```
for i in range(n):
    print("Thread:", current_thread().name)
    print(msg)
# Create and start a worker thread
t1 = Thread(target=display, kwargs={'n': 4, 'msg': "Hello World!"})
t1.start()
# Main thread continues in parallel
for i in range(4):
    print("Thread:", current_thread().name)
    print("welcome")
```

Sample Output (order varies):

```
Thread: Thread-1 (display)
Thread: MainThread
Hello World!
welcome
Thread: Thread-1 (display)
Thread: MainThread
Hello World!
welcome
Thread: Thread-1 (display)
Thread: MainThread
Hello World!
welcome
```

Thread execution order is unpredictable, as it depends on the OS thread scheduler. In this program, the main thread prints "welcome" while the worker thread **t1** prints "Hello World!" by executing `display()`. The `current_thread().name` in both threads prints the thread details. The print order is unpredictable because the Operating system decides when to switch threads, so each run produces a different order. The output clearly shows interleaved statements because both threads execute concurrently. The operating system switches rapidly between the two threads. On a multi-core CPU, true parallel execution may occur; on single-core systems, threads share time slices.

8.3.2 Thread Naming

Each thread in Python has a unique name and is stored in the `.name` attribute, which is useful for debugging. By default, the naming conventions are "MainThread" for the main thread, and "Thread-N" (e.g., Thread-1, Thread-2) for worker threads. Custom names can be assigned, and even the main thread name can be changed to help in debugging.

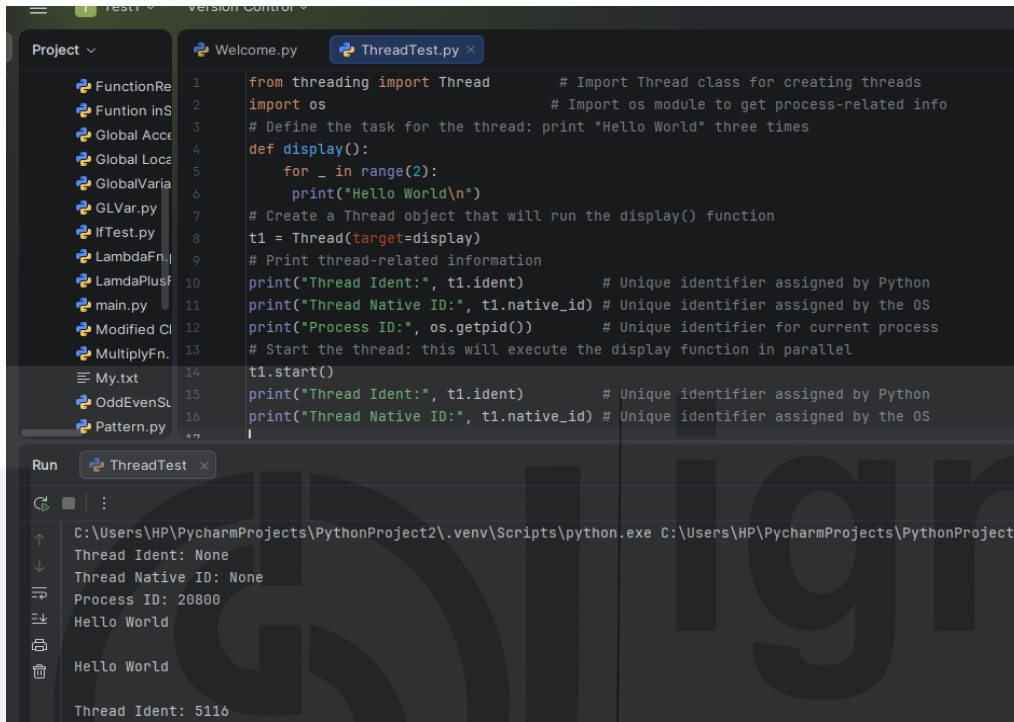
```
t1.name = "Worker-1"
current_thread().name = "NewMainThread"
```

8.3.3 Thread Identifiers

Each thread in Python has two identifiers, namely the Thread Identifier and the Native Identifier. The Thread Identifier (`ident`) is a unique positive number the

Python interpreter assigns when the thread starts. It is read-only and specific to a process. The Native Identifier (`native_id`) is a unique ID assigned at the time of thread creation by the operating system. To be very specific: “The Thread Identifier (`ident`) is a unique non-zero integer that the Python interpreter assigns when a thread starts. It is read-only and specific to a process.”

We can also retrieve a Process ID (PID) using the `os.getpid()` function.



```

1  from threading import Thread      # Import Thread class for creating threads
2  import os                        # Import os module to get process-related info
3  # Define the task for the thread: print "Hello World" three times
4  def display():
5      for _ in range(2):
6          print("Hello World\n")
7  # Create a Thread object that will run the display() function
8  t1 = Thread(target=display)
9  # Print thread-related information
10 print("Thread ID:", t1.ident)      # Unique identifier assigned by Python
11 print("Thread Native ID:", t1.native_id) # Unique identifier assigned by the OS
12 print("Process ID:", os.getpid())  # Unique identifier for current process
13 # Start the thread: this will execute the display function in parallel
14 t1.start()
15 print("Thread ID:", t1.ident)      # Unique identifier assigned by Python
16 print("Thread Native ID:", t1.native_id) # Unique identifier assigned by the OS

```

Run Console Output:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2
Thread ID: None
Thread Native ID: None
Process ID: 20800
Hello World
Hello World
Thread ID: 5116

```

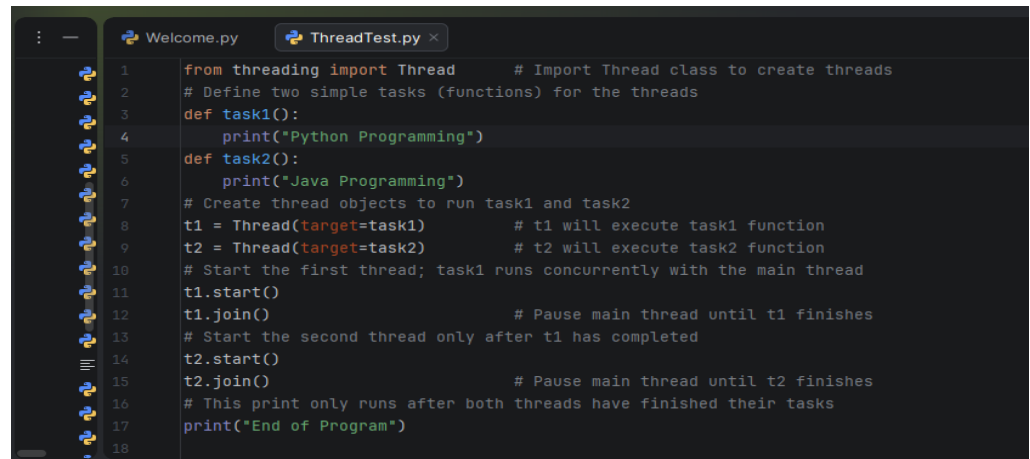
8.3.4 Useful Thread Functions

The threading module provides several built-in functions:

- `is_alive()` – check if a thread is still running.
- `main_thread()` – returns the main thread object.
- `active_count()` – returns number of active threads.
- `enumerate()` – list of all running threads.
- `get_native_id()` – returns the native ID of the current thread.

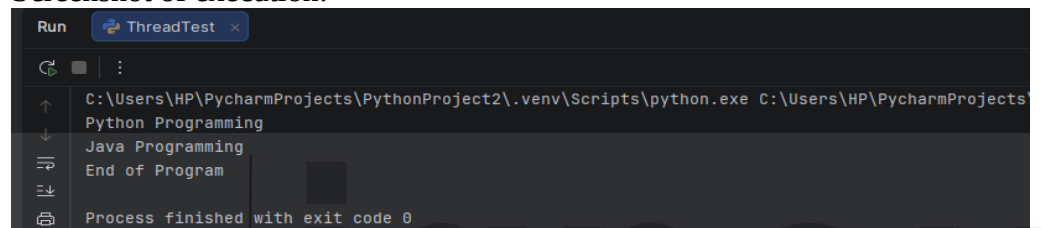
8.3.5 The `join()` Method

When multiple threads have dependencies, output may need to be synchronized. The `join()` method ensures one thread completes before another begins execution.



```
1 from threading import Thread # Import Thread class to create threads
2 # Define two simple tasks (functions) for the threads
3 def task1():
4     print("Python Programming")
5 def task2():
6     print("Java Programming")
7 # Create thread objects to run task1 and task2
8 t1 = Thread(target=task1) # t1 will execute task1 function
9 t2 = Thread(target=task2) # t2 will execute task2 function
10 # Start the first thread; task1 runs concurrently with the main thread
11 t1.start()
12 t1.join() # Pause main thread until t1 finishes
13 # Start the second thread only after t1 has completed
14 t2.start()
15 t2.join() # Pause main thread until t2 finishes
16 # This print only runs after both threads have finished their tasks
17 print("End of Program")
18
```

Screenshot of execution:



```
Run ThreadTest x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects
Python Programming
Java Programming
End of Program
Process finished with exit code 0
```

In the example, the `t1.join()` confirms task2 begins only after task1 finishes, and `t2.join()` confirms the main thread executes last. Without joins, the output order would be random, and in fact the order varies based on the thread schedule.

8.3.6 Daemon Threads

Python supports a special type of thread called a daemon thread. Unlike user threads, which perform main computation, the daemon thread runs in the background and is designed to provide a monitoring service such as log writers, cache managers, etc. A daemon thread automatically stops when all user threads have terminated and is useful for background tasks that should not prevent program exit.

```

1 from threading import Thread, current_thread # To manage threads and get their names
2 import time # For sleep/delay
3 # Daemon thread: repeatedly prints monitoring message every second
4 def monitor():
5     while True:
6         print(f"[{current_thread().name}] Monitoring...")
7         time.sleep(1) # Wait for 1 second before next message
8 # Worker thread: prints a greeting message 2 times
9 def display():
10     for _ in range(2):
11         print(f"[{current_thread().name}] Hello World!")
12         time.sleep(1) # Wait for 1 second between greetings
13 # Create a daemon thread called 'Logger' for monitoring
14 daemon_thread = Thread(target=monitor, name="Logger", daemon=True)
15 # Create a worker thread called 'Worker1' for greetings
16 worker_thread = Thread(target=display, name="Worker1")
17 # Start both threads
18 daemon_thread.start()
19 worker_thread.start()
20 # Meanwhile, the main thread prints its own welcome message twice
21 for _ in range(2):
22     print(f"[{current_thread().name}] welcome")
23     time.sleep(1) # Wait for 1 second between messages
24 # Wait for the worker thread to finish before ending the program
25 worker_thread.join()
26 print("Main thread exiting... Daemon stops automatically.")

```

Screenshot of execution:

```

Run ThreadTest
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects
[Logger] Monitoring...
[Worker1] Hello World!
[MainThread] welcome
[Worker1] Hello World!
[Logger] Monitoring...
[MainThread] welcome
[Logger] Monitoring...
Main thread exiting... Daemon stops automatically.
Process finished with exit code 0

```

In the program, a daemon thread (Logger) runs continuously but stops once the worker and main exit and the program terminates without waiting for the infinite daemon loop.

Check Your Progress – 1

1. What is the primary difference between a process and a thread in terms of memory?

.....

.....

.....

.....

2. Describe a situation where multiple threads could introduce errors if not coordinated.

.....

.....

.....

.....

3. What is the Global Interpreter Lock (GIL), and how does it impact threading in Python?

.....

.....

.....

.....

4. What happens if you call join() before start()?

.....

.....

.....

.....

5. Describe the thread lifecycle. Explain how a thread moves between these states, with a suitable example.

.....

.....

.....

.....

6. When should daemon threads be used instead of non-daemon threads? Explain with an example where a daemon thread is appropriate, and state why a normal thread is not preferred in that case.

.....

.....

.....

.....

8.4 THREAD SYNCHRONIZATION

The multithreading concepts allow multiple threads to run at the same time. But when multiple threads access shared data concurrently, such as a global variable, file, or buffer, without proper coordination, it can corrupt data.

This leads to:

- Race conditions – Results depend on the timing of thread execution.
- Deadlocks – Threads block each other forever.
- Starvation – A thread never gets CPU access due to scheduling or locks.

Python provides synchronization tools to prevent data corruption.

- Locks (Mutual Exclusion)
- Reentrant Locks (RLock)
- Condition Variables
- Semaphores

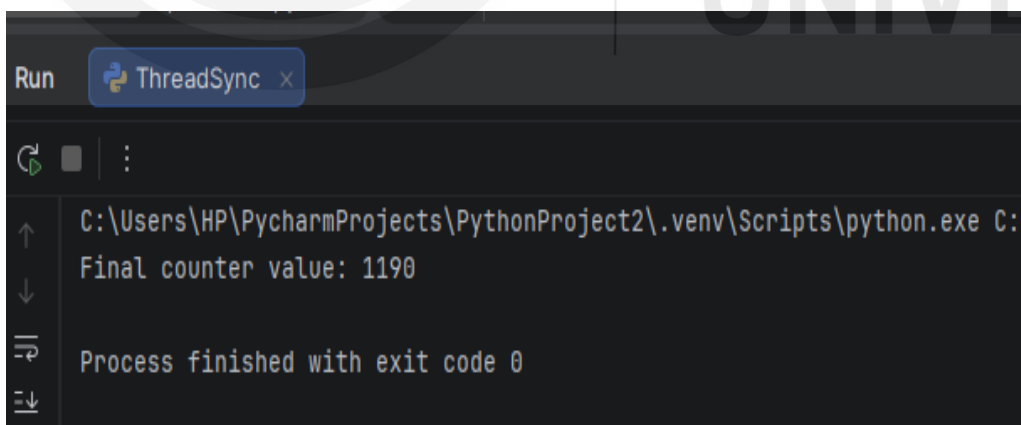
8.4.1 Race Condition

When multiple threads access shared data without coordination, race conditions may occur. Python provides synchronization tools to prevent data corruption. Let's take a simple shared variable counter example that increments via two threads. The example is set up to show how different synchronization tools help in solving the problem.

```
from threading import Thread
import time
counter = 0 # Shared resource
def increment(n):
    global counter
    for _ in range(n):
        temp = counter      # Read the current value
        temp += 1          # Increment it
        time.sleep(0)       # Yield thread context (increases chance of context
switch)
        counter = temp      # Write back
# Two threads increment the same counter
t1 = Thread(target=increment, args=(1000,))
t2 = Thread(target=increment, args=(1000,))
t1.start()
t2.start()
t1.join()
t2.join()
print("Final counter value:", counter)
```

Expected output: 2000

Actual output (race condition): Less than 2000



```
Run ThreadSync x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:
Final counter value: 1190
Process finished with exit code 0
```

In this example, two threads increment a shared counter variable 1000 times. Both threads modify counter *concurrently* without any synchronization. Because the thread execution overlaps, both may read and write the variable at nearly the same time, and without synchronization the two threads might read the same old value of counter and overwrite each other's update. This causes some increments to be "lost", so the final output is often less

than 2000, leading to a race condition. To overcome the race conditions, the proper synchronization is necessary for correct multithreaded programming.

8.4.2 Locks (Mutexes)

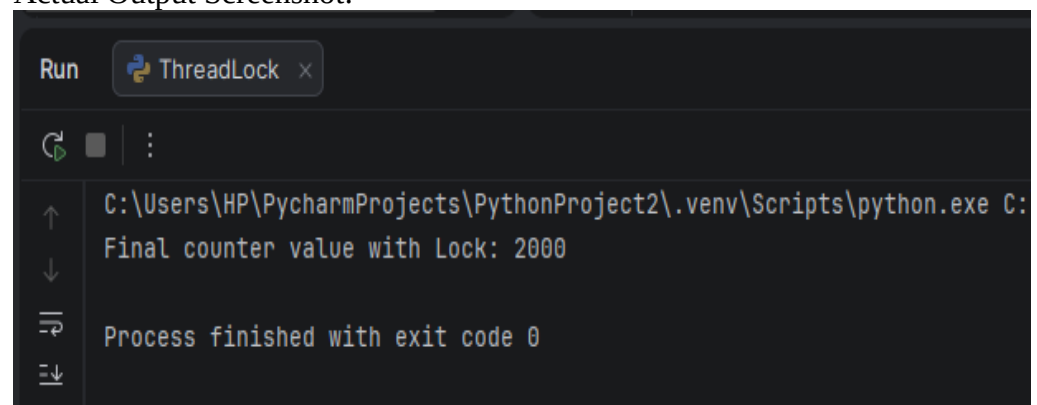
A Lock ensures mutual exclusion where only one thread at a time enters the critical section to modify the shared resource and guarantees correctness:

```
from threading import Thread, Lock
import time
counter = 0      # Shared resource to increment
lock = Lock()    # Lock object to synchronize access
# Function to safely increment the counter 'n' times
def safe_increment(n):
    global counter
    for _ in range(n):
        with lock:      # Acquire lock before modifying counter
            counter += 1 # Critical section: only one thread modifies counter at
                        # a time
            time.sleep(0) # Yield thread context to increase chance of thread
                        # switching
# Create two threads that safely increment the shared counter
t1 = Thread(target=safe_increment, args=(1000,))
t2 = Thread(target=safe_increment, args=(1000,))
# Start threads
t1.start()
t2.start()
# Wait until both threads finish
t1.join()
t2.join()
# Print the final counter value
print("Final counter value with Lock:", counter)
```

Expected Output:

Final counter value with Lock: 2000

Actual Output Screenshot:



```
Run ThreadLock x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:
Final counter value with Lock: 2000
Process finished with exit code 0
```

In this example, the lock enforces mutual exclusion, and thus the synchronization through the lock mechanism will provide the Result 2000 as

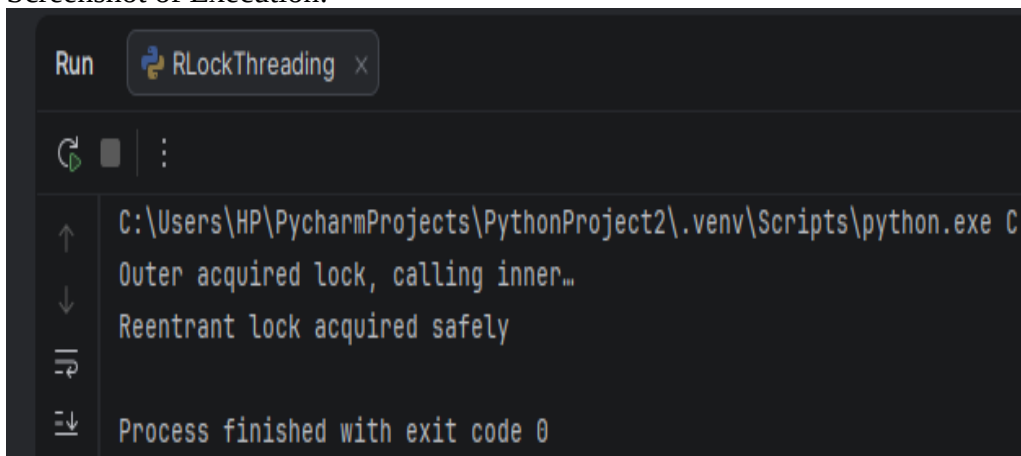
expected. The `with lock:` statement automatically acquires the lock before the block and releases it afterwards. The Lock ensures that only one thread increments the counter at a time, preventing race conditions. This approach properly protects the shared resource and prevents data corruption in multithreading scenarios. But in case the functions call each other and both acquire the same lock, they may be stuck in a deadlock.

8.4.3 Reentrant Locks (RLock)

Normally, a regular Lock cannot be acquired multiple times by the same thread; when a function holding a lock calls another function that also needs the *same lock*, a deadlock occurs with a normal lock. To avoid the deadlock condition an RLock can be used, which allows the same thread to acquire the lock multiple times safely. This is useful when a function holding a lock calls another function that also needs the same lock. The RLock lets the same thread acquire the lock multiple times safely, preventing deadlocks in recursive scenarios or nested locking.

```
from threading import Thread, RLock
lock = RLock()      # Create a reentrant lock object
counter = 0         # Shared variable (not used here but for example)
# Function that acquires the lock and calls another function which also
# acquires the same lock
def outer_task():
    with lock:       # Acquire the reentrant lock
        print("Outer acquired lock, calling inner...")
        inner_task() # Call inner_task while holding the lock
def inner_task():
    with lock:       # Re-acquire the lock safely (same thread can enter multiple
times)
        print("Reentrant lock acquired safely")
# Create and start a thread that runs outer_task
Thread(target=outer_task).start()
```

Screenshot of Execution:



```
Run RLockThreading x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:
Outer acquired lock, calling inner...
Reentrant lock acquired safely
Process finished with exit code 0
```

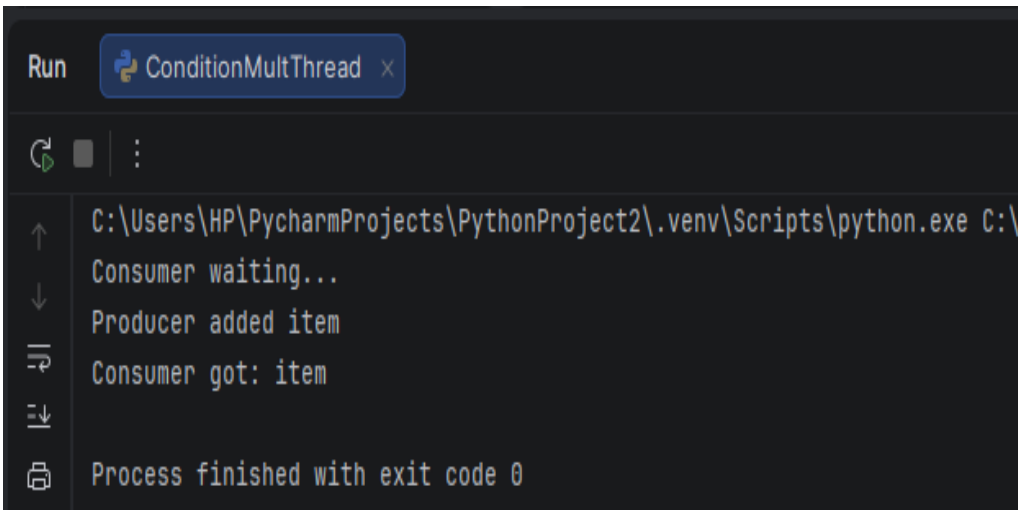
In the above example the `outer_task()` acquires the lock once, then calls `inner_task()`, which re-acquires the lock without blocking. If it were called without an `RLock`, `inner_task()` would block forever, waiting for itself. But, with `RLock`, the thread is “recognized” and allowed to re-enter since it already owns the lock. It allows the Lock with safe recursion. This pattern is useful in complex programs where functions may indirectly call each other while requiring locking

8.4.4 Condition Variables

Sometimes threads need to wait for a condition before taking action. A consumer waits until a producer notifies it about the change. A condition allows a thread to wait for a specific state change. It is very useful for producer-consumer designs, as conditions let threads notify each other through cooperation, and a thread can wait for a specific condition.

```
from threading import Thread, Condition
condition = Condition() # Create a Condition object to synchronize producer
                        & consumer
buffer = []             # Shared resource (buffer) where items are stored
# Producer thread function
def producer():
    with condition:      # Acquire the condition's lock
        buffer.append("item") # Add item to the buffer
        print("Producer added item")
        condition.notify()   # Notify one waiting consumer (if any) that an item
                              is available
# Consumer thread function
def consumer():
    with condition:      # Acquire the condition's lock
        while not buffer: # If buffer is empty, wait until notified
            print("Consumer waiting...")
            condition.wait() # Release lock and wait for notification from
                              producer
        item = buffer.pop() # Remove item from buffer after being notified
        print("Consumer got:", item)
# Start the consumer and producer threads
Thread(target=consumer).start()
Thread(target=producer).start()
```

Screenshot of Execution of the above Program:



```

Run ConditionMultThread x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\
Consumer waiting...
Producer added item
Consumer got: item
Process finished with exit code 0

```

In this example, the consumer thread acquires the condition lock and checks the buffer. If the buffer is empty, it calls `condition.wait()` to release the lock so the producer can acquire it and suspends the consumer until it is notified. The producer thread adds an item and calls `condition.notify()` to wake up a waiting consumer. Once the consumer is notified, it reacquires the lock, rechecks the buffer using a while loop, and consumes the item. This approach ensures proper coordination between producers and consumers without busy waiting. This model efficiently synchronizes threads sharing a resource using condition variables.

8.4.5 Semaphores

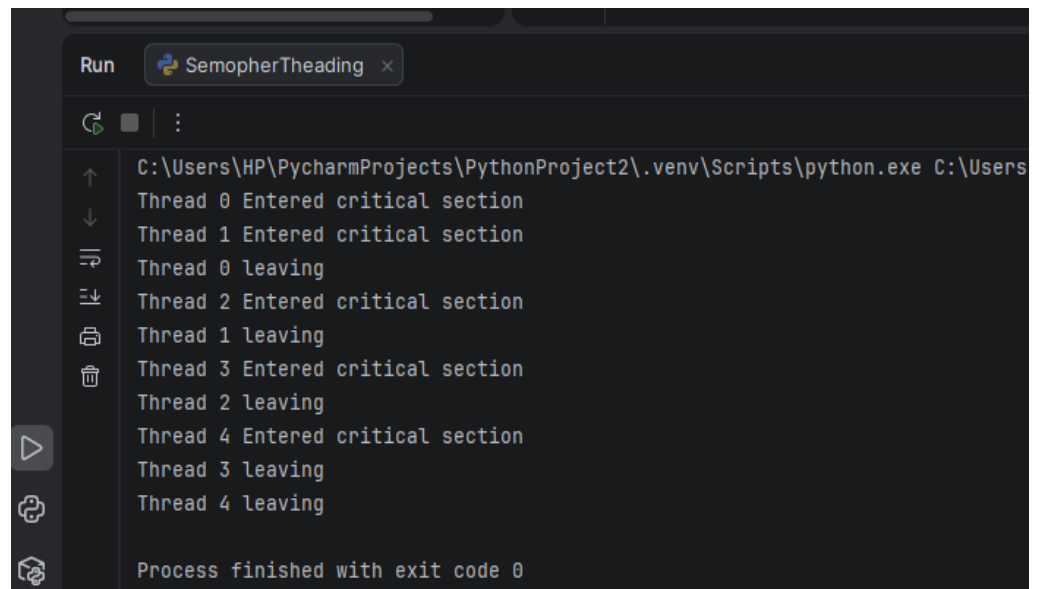
A Semaphore allows up to *n* threads inside a block to access a resource at the same time by limiting concurrent access to resources.

```

from threading import Thread, Semaphore
import time
sem = Semaphore(2) # Create a semaphore allowing up to 2 threads
concurrently in critical section
# Worker function that tries to enter critical section limited by semaphore
def task(i):
    with sem: # Acquire semaphore before entering critical section, blocks if
limit reached
        print("Thread", i, "Entered critical section")
        time.sleep(5) # Simulate some work inside the critical section
        print("Thread", i, "leaving") # Release semaphore automatically when
block ends
# Launch 5 threads running task with different IDs
for i in range(5):
    Thread(target=task, args=(i,)).start()

```

Screenshot of Execution of the above program:



```
Run SemopherTheading x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users
Thread 0 Entered critical section
Thread 1 Entered critical section
Thread 0 leaving
Thread 2 Entered critical section
Thread 1 leaving
Thread 3 Entered critical section
Thread 2 leaving
Thread 4 Entered critical section
Thread 3 leaving
Thread 4 leaving
Process finished with exit code 0
```

In this example, the semaphore `sem` allows at most 2 threads to enter the critical region at the same time. When a thread enters the with `sem: block`, it tries to acquire the semaphore. But only if fewer than 2 threads hold the semaphore, there is a success; otherwise, it blocks and waits. After `time.sleep(5)`, the thread finishes and releases the semaphore automatically. Basically, the 5 threads are started almost simultaneously, but only 2 threads will be inside the critical section concurrently; the others wait until a permit is released. This is useful to limit concurrent access to a limited resource, like connections or bandwidth.

This example shows effective use of Semaphores to control concurrent thread execution.

Check Your Progress – 2

1. What is a race condition, and how can it be avoided?

.....

.....

.....

.....

2. What is the difference between a Lock and an RLock?

.....

.....

.....

.....

3. How do semaphores differ from locks?

.....

.....

.....

.....

4. Explain the producer-consumer problem. How do Condition variables solve it? Show how a producer adds items and a consumer waits for items without wasting CPU time.

.....

.....

.....

.....

5. What causes deadlock and starvation in programs with many threads? How are they different? How can we avoid deadlock when using more than one lock?

.....

.....

.....

.....

8.5 ADVANCED THREADING TECHNIQUES

While basic functionality like creating, naming, and joining threads covers most needs, Python's threading module also supports advanced threading patterns for real-world applications that improve scaling and maintainability.

These include:

1. Thread Pools, for efficient task management.
2. Daemon vs Non-Daemon Threads, for foreground vs background logic.
3. Thread-Safe Queues, for producer-consumer patterns.
4. Timer Threads, for delayed task execution.

8.5.1 Thread Pools (via `concurrent.futures`)

Let us consider a scenario where users want to download 100 webpages or process 100 data files, which would require creating 100 separate threads manually using threading. However, manually managing each thread is very inefficient. Rather than manually starting and joining multiple threads, Python provides thread pools for managing worker threads more efficiently. Instead of creating a new thread per job, a pool of reusable worker threads is created. Jobs are then distributed among these workers. This reduces overhead, keeps code cleaner, and is ideal when many independent tasks need to be run. Note that the "`concurrent.futures` module in Python provides a high-level interface to run tasks asynchronously using threads or processes".

```
from concurrent.futures import ThreadPoolExecutor
import time
# Simulated task function representing downloading or processing a file
def process_task(task_id):
    print(f"Job {task_id} started") # Indicate start of task
```

```
time.sleep(1)          # Simulate work with a delay (e.g., download time)
print(f"Job {task_id} completed") # Indicate task completion
# Main section that creates and uses a thread pool to manage tasks efficiently
if __name__ == "__main__":
    # Create a thread pool with a fixed number of worker threads (e.g., 5)
    with ThreadPoolExecutor(max_workers=5) as executor:
        # Submit 100 independent tasks to the thread pool
        for i in range(100):
            executor.submit(process_task, i)
```

OUTPUT:

```
Job 0 started
Job 1 started
Job 2 started
Job 3 started
Job 4 started
Job 0 completed
Job 5 started
Job 1 completed
Job 3 completed ...
and so on...
```

In this example, creating 100 individual threads will be resource-intensive and hard to manage; instead, we created a thread pool with 5 workers. The pool reuses these 5 threads to process all 100 jobs efficiently by queuing the jobs and distributing them as workers become available. The function `process_task` simulates work done by each thread, such as downloading or processing and using `ThreadPoolExecutor` simplifies starting, managing, and cleaning up threads while ensuring resource optimization.

Automatic management of workers and are better suited for scenarios with large numbers of small, independent jobs like web scraping, batch file processing, or parallel I/O operations.

8.5.2 Daemon vs. Non-Daemon Threads

Some threads are critical and must finish before the program exits, like saving a file, processing data, etc. Others are background service threads like monitoring CPU usage or writing logs, which should not block program termination.

Python allows you to set a thread as:

- **User (non-daemon) Threads:** Perform essential computation; the program waits for them to complete.
- **Daemon Threads:** Run in the background and terminate automatically when the main program exits.

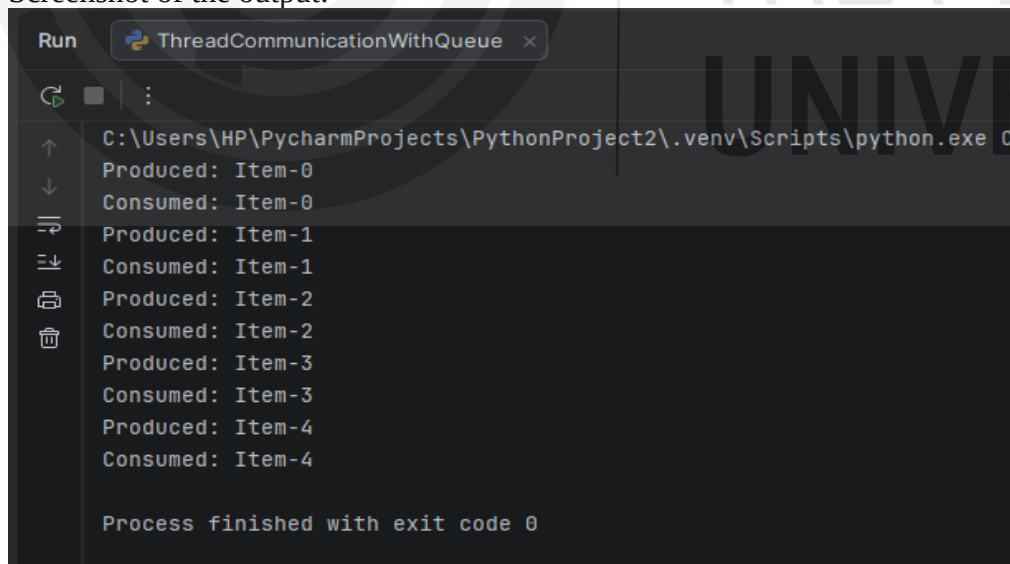
8.5.3 Thread Communication with Queue

In any producer-consumer problem, manually synchronizing producers (threads creating data) and consumers (threads using data) is difficult. We may need

locks, conditions, and careful ordering to avoid lost or duplicate data. Python's `queue.Queue` is thread-safe and avoids manual locks for producer-consumer designs by providing a thread-safe FIFO queue with automatic locking.

```
from threading import Thread
from queue import Queue
import time
q = Queue() # Create a thread-safe queue for communication between
producers and consumers
# Producer function that adds items to the queue
def producer():
    for i in range(5):
        item = f"Item-{i}" # Create an item (here a simple string)
        q.put(item)        # Put item into the queue (thread-safe insertion)
        print("Produced:", item)
        time.sleep(2)      # Simulate some delay between producing items
# Consumer function that removes and processes items from the queue
def consumer():
    while True:
        item = q.get()     # Wait and get an item from the queue (blocks if empty)
        print("Consumed:", item)
        q.task_done()     # Indicate that processing of an item is complete
# Start the producer thread
Thread(target=producer).start()
# Start the consumer thread as a daemon thread, so it does not block program
exit
Thread(target=consumer, daemon=True).start()
```

Screenshot of the output:



```
Run ThreadCommunicationWithQueue x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:
Produced: Item-0
Consumed: Item-0
Produced: Item-1
Consumed: Item-1
Produced: Item-2
Consumed: Item-2
Produced: Item-3
Consumed: Item-3
Produced: Item-4
Consumed: Item-4
Process finished with exit code 0
```

In this example, the producer creates 5 items and puts them into the queue safely, even with concurrent consumers. The consumer threads continuously wait for items, `q.get()` is blocking if no items are present and processes them as they come. Using a thread-safe `queue.Queue` avoids complicated manual locking for synchronization. The consumer runs as a daemon thread, so when the main program finishes i.e. the producer completes, the consumer thread exits.

automatically. This pattern is effective for coordinating multiple producers and consumers across threads in Python.

8.5.4 Timer Threads

Sometimes, tasks should be executed after a delay, like reminders, scheduled events, or retrying failed requests. Although sleeping in the main program may handle these cases, but it blocks other logic. Instead, Python provides `threading.Timer`, which runs a function in a separate thread after a given timeout.

```
from threading import Timer # Import Timer class for scheduling delayed
execution
# Function to be executed after the delay
def greeting():
    print("Hello after 3 seconds!")
# Create a Timer object that will call 'greeting' after 3 seconds
t = Timer(3, greeting)
# Start the timer thread; 'greeting' will run after 3 seconds without blocking
main thread
t.start()
```

OUTPUT: Hello after 3 seconds!

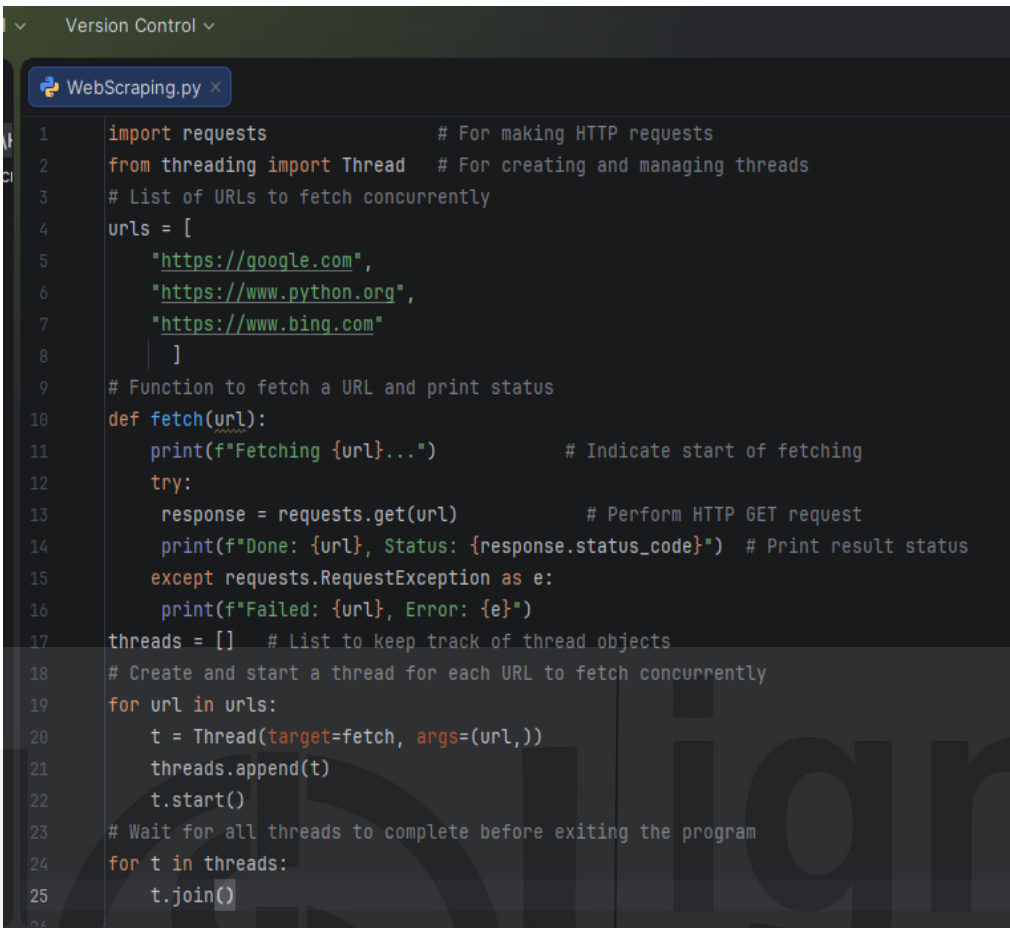
In this example, the `Timer` class runs the `greeting()` in a separate thread after a delay of 3 seconds. The main program continues without blocking during this wait, and after 3 seconds, the greeting function executes automatically in the timer thread. This is useful for scheduling tasks that need to run after a pause or delay without freezing the rest of the program.

8.6 MULTITHREADING WITH REAL-WORLD EXAMPLES

Now, let's explore practical scenarios where threads are commonly used.

8.6.1 Example 1 – Web Scraping (Concurrent Requests)

Fetching web pages sequentially is slow because you wait for server responses. Threads let you fetch multiple pages in parallel by retrieving multiple URLs at once.



```

1  import requests          # For making HTTP requests
2  from threading import Thread # For creating and managing threads
3  # List of URLs to fetch concurrently
4  urls = [
5      "https://google.com",
6      "https://www.python.org",
7      "https://www.bing.com"
8  ]
9  # Function to fetch a URL and print status
10 def fetch(url):
11     print(f"Fetching {url}...")          # Indicate start of fetching
12     try:
13         response = requests.get(url)      # Perform HTTP GET request
14         print(f"Done: {url}, Status: {response.status_code}") # Print result status
15     except requests.RequestException as e:
16         print(f"Failed: {url}, Error: {e}")
17 threads = [] # List to keep track of thread objects
18 # Create and start a thread for each URL to fetch concurrently
19 for url in urls:
20     t = Thread(target=fetch, args=(url,))
21     threads.append(t)
22     t.start()
23 # Wait for all threads to complete before exiting the program
24 for t in threads:
25     t.join()

```

Screenshot of Execution:



```

Run  WebScraping x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\
Fetching https://google.com...
Fetching https://www.python.org...
Fetching https://www.bing.com...
Done: https://www.python.org, Status: 200
Done: https://google.com, Status: 200
Done: https://www.bing.com, Status: 200

Process finished with exit code 0

```

In this example, the script fetches multiple webpages in parallel using threads, improving overall time compared with sequential fetching. Each URL is handled by a dedicated thread running the fetch function independently. Using `t.join()` ensures the main program waits for all fetch operations to complete before finishing. This is suitable for I/O-bound tasks like network requests where threads can run concurrently during wait times without being blocked by Python's Global Interpreter Lock (GIL). In this example, the threads reduce total runtime by overlapping network waits.

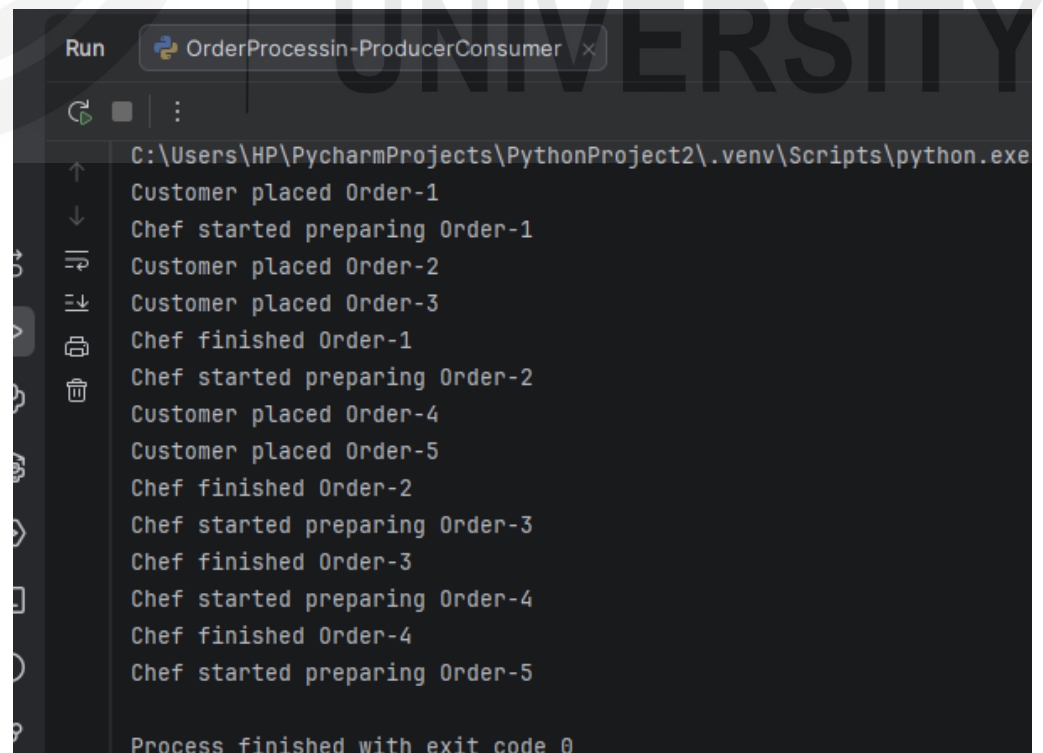
8.6.2 Example 2 – Producer–Consumer Problem

Here, the producer-consumer pattern uses threads and a queue in a restaurant order-preparation scenario. This example demonstrates a producer preparing

orders and a consumer consuming them. Without coordination, consumers may try to consume before items exist. The `queue.Queue` solves this classic concurrency problem:

```
from threading import Thread
from queue import Queue
import time
orders_queue = Queue() # Queue to hold orders
# Producer function: Customer placing orders
def customer():
    for order_num in range(1, 6):
        order = f"Order-{order_num}"
        orders_queue.put(order) # Customer places order in the queue
        print(f"Customer placed {order}")
        time.sleep(0.7) # Time between orders
# Consumer function: Chef preparing orders
def chef():
    while True:
        order = orders_queue.get() # Chef picks up next order from queue
        print(f"Chef started preparing {order}")
        time.sleep(2) # Time taken to prepare the order
        print(f"Chef finished {order}")
        orders_queue.task_done() # Mark order as completed
# Start threads: one customer placing orders and one chef preparing
Thread(target=customer).start()
Thread(target=chef, daemon=True).start()
# Allow time for all orders to be processed before main thread exits
time.sleep(10)
```

Screenshot of execution of program:



```
Run OrderProcessin-ProducerConsumer
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe
Customer placed Order-1
Chef started preparing Order-1
Customer placed Order-2
Customer placed Order-3
Chef finished Order-1
Chef started preparing Order-2
Customer placed Order-4
Customer placed Order-5
Chef finished Order-2
Chef started preparing Order-3
Chef finished Order-3
Chef started preparing Order-4
Chef finished Order-4
Chef started preparing Order-5
Process finished with exit code 0
```

In this example, the customer thread produces orders at intervals and places them in the queue. The chef thread simulates preparation time with `sleep()`. The queue ensures safe communication and ordering between customers and chefs. The chef is a daemon thread running continuously in the background but won't prevent the main program from exiting once all orders are done. This real-world analogy makes the producer-consumer concept tangible and easier to grasp.

8.6.3 Example 3 – Logging in Background

A daemon thread constantly writes logs while the main task executes. Main tasks should not be blocked by repetitive background tasks like logging; thus, a daemon thread runs logging while the main program continues independently.

```
import time
from threading import Thread, current_thread
# Daemon thread function: continuously writes logs every second
def logger():
    while True:
        print(f"[{current_thread().name}] Writing logs...")
        time.sleep(1) # Sleep for 1 second between log writes
# Main task function: simulates a task with three steps, each taking 2 seconds
def main_task():
    for i in range(3):
        print(f"Main task step {i}")
        time.sleep(2) # Simulate time-consuming work per step
# Start the logger as a daemon thread named "LogDaemon"
Thread(target=logger, name="LogDaemon", daemon=True).start()
# Run the main task on the main thread
main_task()
# After the main task completes, print exit message; daemon thread stops
# automatically
print("Program exiting (daemon log thread stops).")
```

OUTPUT:

```
[LogDaemon] Writing logs...
Main task step 0
[LogDaemon] Writing logs...
[LogDaemon] Writing logs...
Main task step 1
[LogDaemon] Writing logs...
Main task step 2
[LogDaemon] Writing logs...
[LogDaemon] Writing logs...
Program exiting (daemon log thread stops).
```

In this example, the logger function runs in a separate daemon thread, printing log messages every second indefinitely. The `main_task` simulates work on the main thread, with 3 steps of 2 seconds each. The logger thread is a daemon, so,

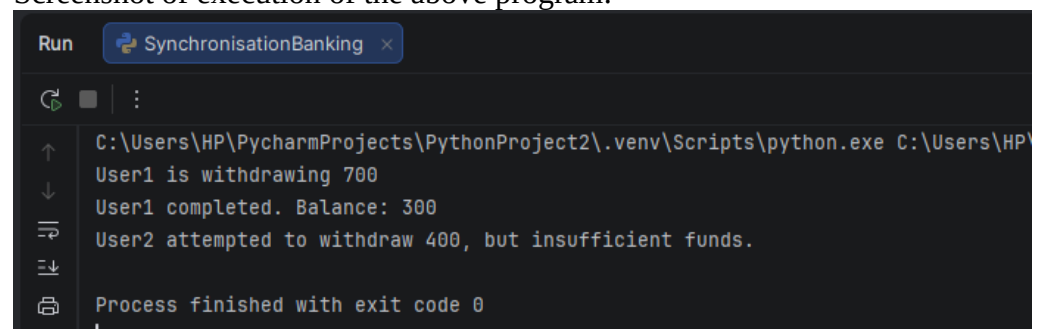
it will automatically stop once the main thread completes and the program exits. The output lines from the daemon and main thread are interleaved, demonstrating concurrent execution.

8.6.4 Example 4 – Simulation (Bank Transactions with Lock)

Locks ensure mutual exclusion where two users withdrawing money simultaneously must be synchronized to prevent overdrawing.

```
from threading import Thread, Lock
import time
balance = 1000      # Shared bank account balance
lock = Lock()       # Lock to synchronize access to balance
# Function to simulate withdrawing money safely
def withdraw(name, amount):
    global balance
    with lock:        # Acquire lock before checking and modifying balance
        if balance >= amount:
            print(f"{name} is withdrawing {amount}")
            time.sleep(1) # Simulate processing delay
            balance -= amount
            print(f"{name} completed. Balance: {balance}")
        else:
            print(f"{name} attempted to withdraw {amount}, but insufficient
funds.")
# Create two threads representing two users trying to withdraw simultaneously
threads = [
    Thread(target=withdraw, args=("User1", 700)),
    Thread(target=withdraw, args=("User2", 400))
]
# Start both withdrawal threads
for t in threads:
    t.start()
# Wait for both threads to complete
for t in threads:
    t.join()
```

Screenshot of execution of the above program:



```
Run SynchronisationBanking x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\
User1 is withdrawing 700
User1 completed. Balance: 300
User2 attempted to withdraw 400, but insufficient funds.
Process finished with exit code 0
```

In this example, two threads try to withdraw money from the shared balance. Without locks, both users might wrongly succeed. With synchronization, safety is ensured. The lock ensures that only one thread accesses the balance at a time,

preventing inconsistent or corrupted balance values. Without the lock, both could check the balance before any withdrawal. The `time.sleep(1)` simulates a delay, which increases the chances of thread interference if no lock is used. With proper locking, one user's withdrawal completes before the other begins, preventing race conditions and ensuring accurate balance updates. This example clearly shows the importance of synchronization when multiple threads share and modify common resources.

Check Your Progress – 3

1. What is the main advantage of using a Thread Pool instead of creating individual threads manually? Provide an example of a situation where this would be beneficial.

2. Differentiate between Daemon threads and Non-Daemon threads. Why is it a bad idea to make a critical computation thread a daemon?

3. Explain why using a Queue makes the Producer–Consumer problem easier to implement compared to using Locks and Conditions directly.

4. Why is threading helpful in web scraping?

5. In logging examples, why should the log writer thread be a daemon thread?

8.7 SUMMARY

This unit explores Python multithreading by distinguishing between threads and processes and explaining GIL implications. It details thread creation and management, covering names, IDs, `join()` calls, and daemon configurations. To handle race conditions, key synchronization mechanisms—including Locks, RLocks, Conditions, and Semaphores—are comprehensively covered. Towards the end of the unit, it explains advanced techniques like thread pools, queues, and timer threads. Finally, it demonstrates practical applications through real-world scenarios such as web scraping, producer–consumer workflows, simulations, and logging.

8.8 SOLUTIONS / ANSWERS TO CHECK YOUR PROGRESS

Check Your Progress – 1:

1. A process has its own independent address space and memory; its data is not directly visible to other processes.

A thread runs inside a process and shares that process's memory space (heap, global variables, code), which makes inter-thread communication easy but also introduces the risk of data conflicts if not synchronized.

2. A classic example is multiple threads incrementing a shared counter without synchronization.

Each thread reads the current value, increments it, and writes it back; if two threads interleave these steps, increments can be “lost,” so the final value is less than expected, which demonstrates a race condition.

3. The Global Interpreter Lock (GIL) is a mutex in Python that allows only one thread at a time to execute Python bytecode, even on multicore processors. Because of the GIL, CPU-bound threads do not run in true parallel and gain little or no speedup, while I/O-bound threads (performing network, file, or database I/O) can still benefit significantly since the GIL is released during blocking I/O operations.

4. If `join()` is called on a thread before `start()`, the call returns immediately because the thread has not been started and therefore is already considered “not alive.”

In normal usage, `join()` is called after `start()` to block until the thread has finished; calling it before `start()` provides no synchronization benefit and is a logical error in program design, even though it does not raise an exception.

5. The document describes five conceptual states for a thread:

- **New:** The Thread object is created but `start()` has not yet been called.
- **Runnable:** After calling `start()`, the thread is ready to run and waiting for CPU time.
- **Running:** The thread is actively executing its target function.

- Blocked/Waiting: The thread is temporarily suspended, for example while sleeping, waiting for I/O, or waiting to acquire a lock or condition.
- Terminated: The target function completes or exits due to an error; the thread cannot be restarted.

Example illustrating transitions:

```
from threading import Thread
import time

def worker():
    print("Thread started")    # Running
    time.sleep(1)             # Running → Blocked/Waiting
    print("Thread resuming")  # Blocked/Waiting → Running
    # When function returns, thread → Terminated

t = Thread(target=worker)    # New
print("Thread created (New)")

t.start()                   # New → Runnable (then Running when scheduled)
print("Thread started (Runnable/Running)")

t.join()                    # Wait until Terminated
print("Thread has terminated")
```

The OS scheduler moves the thread between Runnable and Running, blocking calls like `sleep()` or waiting on locks move it to Blocked/Waiting, and natural completion moves it to Terminated.

6. Daemon threads are appropriate for background tasks that should not prevent the program from exiting, such as logging, monitoring, or housekeeping.

Non-daemon (normal) threads are used for essential work that must finish before the application terminates, such as saving data, completing a transaction, or finishing a computation.

Example from the document: a background logger thread that continuously writes “[Logger] Writing logs...” while the main task runs is started as a daemon:

```
from threading import Thread
import time
def logger():
    while True:
        print("[Logger] Writing logs...")
        time.sleep(1)

def main_task():
```

```
for i in range(3):  
    print(f"Main task step {i}")  
    time.sleep(2)
```

```
Thread(target=logger, name="LogDaemon", daemon=True).start()  
main_task()  
print("Program exiting (daemon log thread stops).")
```

Here, the logger is a daemon because logging is secondary; the program should exit as soon as `main_task()` finishes, even if the logger loop would otherwise continue indefinitely.

If the logger were a normal (non-daemon) thread, the program would not terminate until the logging loop stopped explicitly, which is not desirable for this kind of background service.

Check Your Progress – 2

1. A race condition happens when two or more threads try to read and change the same shared data at the same time. The final result depends on which thread finishes first, so the program gives wrong answers. It can be avoided by using synchronization tools like Lock to make sure only one thread can access the shared data at a time. Wrap the critical code (read + change + write) inside a lock, so other threads wait until the first thread finishes.

2. Only one thread can take it at a time in Locks. If the same thread tries to take it again without first releasing it, the program gets stuck (deadlock).

The same thread can take it multiple times without getting stuck in RLock (Reentrant Lock). It keeps a count and only releases when the thread calls release the same number of times it took the lock. It is a good approach for nested function calls that all need the same lock.

3. A lock is like a single key - only 1 thread can enter the protected area at a time.

A semaphore is like multiple keys - it allows up to N threads (set by the counter) to enter together. When counter reaches 0, other threads wait until some thread leaves and frees a "key".

4. In the Producer-Consumer Problem, the Producer threads make items and put them in a shared buffer. Consumer threads take items from the buffer to process. Problems without synchronization:

- Producer fills buffer when it's already full (overwrite data)
- Consumer tries to take items when buffer is empty (crash/error)

Condition Variables Solution: Combines a lock + wait/notify so threads sleep efficiently instead of busy-checking the buffer repeatedly.

The producer adds items and a consumer waits for items without wasting CPU time through the process:

- i. Producer: Checks if the buffer has space. If full, calls `condition.wait()` , which releases the lock and sleeps. When woken, adds item and calls `condition.notify()` to wake consumer.
- ii. Consumer: Checks if buffer has items. If empty, calls `condition.wait()` which releases lock and sleeps. When woken, takes item and calls `condition.notify()` to wake producer.
- iii. No CPU waste: `wait()` puts the thread to sleep (releases CPU). `notify()` wakes exactly when needed.

5. Deadlock Causes: Two+ threads each hold a lock and wait for a lock the other thread holds. Neither can move forward. All threads are completely stuck and can only be recovered by a restart.

Starvation Causes: One thread never gets the CPU/lock because other threads always get it first due to either scheduling bias or priority issues.

We may avoid Deadlock using Multiple Locks:

- i. Fixed Order Rule: Always grab locks in the same order (Lock1→Lock2→Lock3). No circular waits possible.
- ii. Timeouts: Try to grab a lock for a few seconds only; give up if it fails.
- iii. Higher-level tools: Use Queue or RLock instead of multiple locks.

Check Your Progress – 3

1. A Thread Pool reuses a fixed number of worker threads instead of creating and destroying a new thread for every task. This saves CPU/memory overhead, prevents creating too many threads, and simplifies management.

For Example: Web scraping 100 URLs. Instead of creating 100 individual threads (which wastes resources), use a pool of 10 worker threads. As one URL finishes, the worker immediately starts the next task from the queue. Much faster and more efficient than sequential processing or 100 individual threads.

2. Background helper threads that automatically stop when the main program exits are the Daemon threads and are used for non-critical tasks like logging or monitoring. Non-Daemon threads are the essential worker threads. The program waits for them to finish before exiting.

Critical work must be completed fully, so use of non-daemon threads. In case an important calculation (like processing payments or saving data) is performed using a daemon thread, the program might exit before it finishes, and you may end up losing the results.

3. A queue simplifies Producer-Consumer because of the queue. Queue is automatically thread-safe and no manual locks needed. `q.put(item)` safely adds items and blocks if full. `q.get()` safely removes items and blocks if empty, so no busy-waiting. Also, there is Built-in synchronization handles producer/consumer coordination automatically.

Locks + Conditions require manual coding, where manually checking the buffer full/empty and Manual wait()/notify() calls may easily make mistakes with lock ordering or notifications.

4. Web scraping is I/O-bound (waiting for slow network responses), not CPU-bound. Threading helps because:

- i. While one thread waits for Server A to respond, others can request Server B, C, etc.
- ii. GIL is released during network I/O, so multiple threads make real concurrent progress
- iii. Instead of waiting 10 seconds sequentially for 10 URLs, 10 threads might finish in ~2 seconds (overlapping waits)

5. The log writer should be daemon because:

- i. Logging is secondary - main program work is more important
- ii. Endless loop - logger runs while True: writing logs continuously
- iii. Don't block exit - program should exit immediately when main work finishes, even if logger is still writing final entries
- iv. Acceptable loss - missing a few final log lines on shutdown is OK

If log writer is non-daemon, then it may hang forever waiting for logger loop to stop (which never happens).

Document example: Logger daemon prints continuously while main task runs 3 steps, then both exit cleanly together.

8.9 REFERENCES/FURTHER READINGS

Recommended Books

- “Python Cookbook” by David Beazley and Brian K. Jones (3rd Edition)
- “Let Us Python” by Yashavant Kanetkar & Aditya Kanetkar (7th Edition, BPB Publications, 2024)
- “Programming Python” by Mark Lutz (4th Edition)
- “Core Python Programming” by Dr. R. Nageswara Rao (3rd Edition, Dreamtech Press)
- “Python Concurrency with asyncio” by Matthew Fowler, 2022.
- “Advance Core Python Programming” by Meenu Kohli (BPB Publications, 2021)
- “Fluent Python” by Luciano Ramalho (2nd Edition)

Recommended Online Resources

- Python Official Documentation - threading module: <https://docs.python.org/3/library/threading.html>
- Real Python - Threading Guide: <https://realpython.com/intro-to-python-threading/>

- David Beazley on the GIL (Video):
<https://www.youtube.com/watch?v=Obt6Ewi7pVY>
- Python Queue Documentation:
<https://docs.python.org/3/library/queue.html>
- concurrent.futures Documentation:
<https://docs.python.org/3/library/concurrent.futures.html>
- Real Python - Thread Safety: <https://realpython.com/intro-to-python-threading/#thread-safety>
- Python Course - Threading Concepts: https://www.python-course.eu/advanced_python/threading.php

